

ZERO-KNOWLEDGE-ASSUMED EDITION · EXERCISES 1-9

Machine Learning Laboratory

A complete beginner's study guide —
every dataset, every library, every line of
code explained in plain English.

Data Exploration

Data Cleaning

PCA

Decision Trees

Regression

SMOTE

Contents

Getting Started

Absolute Basics (Read This First)	2
Master Library Cheat-Sheet	2

Exercise 1 — Exploring Three Types of Data

1a Tabular Data (Student Marks)	4
1b Text Data (Movie Reviews / NLP Basics)	6
1c Image Data (CIFAR-10 Pictures)	8

Exercise 2 — Data Cleaning

2a Cleaning Employee Data	10
2b Cleaning a Messy Cafe Sales Dataset	12

Core Exercises

3 Data Preparation (Encoding & Scaling)	14
4 Principal Component Analysis (PCA)	17
5 Decision Tree Classifier	19
6 Simple Linear Regression	22
7 Decision Tree Pruning & Evaluation Metrics	25
8 Logistic Regression	28
9 Handling Imbalanced Data with SMOTE	31

Wrap-Up

Plotting Quick Reference	34
Final Exam Summary Table	35
Last-Minute Checklist	35

How to use this guide: Read top to bottom. Later exercises reuse ideas from earlier ones, so don't skip around on your first pass. Every exercise follows the same structure: *What it means* → *Libraries you need* → *Full commented code* → *Plot explained* → *Key terms to memorize*.

Absolute Basics (Read This First)

You know nothing yet — so start here. These words appear in almost every exercise below.

Term	Simple Meaning
Python	The programming language everything below is written in.
Library / Package	A ready-made toolbox of code someone else wrote so we don't reinvent it. We "import" a library to use its tools.
import pandas as pd	"Bring in the pandas toolbox, and let me call it pd for short."
Dataset	A table of data — rows = individual records (e.g., one student), columns = attributes (e.g., age, marks).
DataFrame	What pandas calls a data table once it's loaded into Python. Usually stored in a variable called df.
Feature (X)	An input column used to make a prediction (e.g., "Years of Experience").
Target / Label (y)	The column we are trying to predict (e.g., "Salary").
Model	A mathematical recipe that learns patterns from data so it can make predictions on new, unseen data.
Training	Showing the model examples with the correct answers so it can learn the pattern.
Testing	Checking how well the model learned, using examples it has never seen before.
.fit()	The command that tells a model "learn from this data now."
.predict()	The command that tells a model "now guess the answer for this new data."
Overfitting	When a model memorizes the training data too well (including its noise) and performs badly on new data — like memorizing exam answers instead of understanding the topic.

Master Library Cheat-Sheet

Every library used across all 9 exercises, explained once so you can refer back anytime.

Library	What It Is	Why We Use It
pandas (pd)	A spreadsheet-like toolbox for Python.	Load CSV/Excel files, view rows/columns, clean data, filter, group.
numpy (np)	A toolbox for fast math on lists of numbers ("arrays").	Generate random numbers, calculate square roots, means, etc.
matplotlib.pyplot (plt)	The core plotting/graphing toolbox.	Draw histograms, scatter plots, bar charts, line plots, boxplots.
seaborn (sns)	A "prettier," easier version of matplotlib with built-in sample datasets.	Nicer-looking charts, and instantly loads the built-in titanic dataset.
wordcloud	Turns text into a picture where common words appear bigger.	Visualize which words appear most often in a body of text.
nltk	"Natural Language Toolkit" — a library for working with human	Break text into sentences/words, remove filler words ("stopwords").

language.

pickle	Lets Python save/load objects (like images) to/from a file.	The CIFAR-10 image dataset is saved in "pickle" format, so we need this to open it.
scikit-learn (sklearn)	The main Machine Learning toolbox in Python.	Ready-made models (Decision Tree, Regression), data splitters, scalers, encoders, accuracy checkers.
imbalanced-learn (imblearn)	An add-on to scikit-learn for unbalanced datasets.	Provides SMOTE — a technique that fixes datasets where one class is much rarer than another.

TERMINAL

```
pip install pandas numpy matplotlib seaborn wordcloud nltk scikit-learn imbalanced-learn openpyxl
```

Install everything at once (run this once in a terminal, not in your script).

Exploring Three Types of Data (Table, Text, Image)

What are we doing? Real-world data comes in different shapes. This exercise teaches you to open and inspect three kinds of data:

- **1a — Tabular data** — a normal spreadsheet (rows & columns), like a report card.
- **1b — Text data** — paragraphs of writing, like movie reviews.
- **1c — Image data** — pictures, stored as grids of numbers (pixels).

This whole process is called **Exploratory Data Analysis (EDA)** — a fancy term for "look carefully at your data before doing anything else with it."

Part 1a — Tabular Data (Student Marks)

Dataset: `student_data.csv` — students with a column `G3` (final grade)

Library	Why We Need It
<code>pandas</code>	Load the CSV file and explore rows/columns/statistics.
<code>matplotlib.pyplot</code>	Draw a histogram of the marks.
<code>seaborn</code>	Imported for nicer plot styling (available if needed).

FULL CODE

```

# 1. Load the necessary toolboxes
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# 2. Load the dataset from a CSV file into a table called "df"
df = pd.read_csv(r"D:\SEM_5\ML\datasets\Table_data\student_data.csv")

# 3. Look at the first 10 rows, just to get a feel for the data
print("First 10 Records:")
print(df.head(10))          # .head(10) = "show me the top 10 rows"
print(df.dtypes)            # dtypes = what TYPE of data is in each column

# 4. How big is the table?
rows, cols = df.shape       # .shape gives (number_of_rows, number_of_columns)
print("\nNumber of Rows:", rows)
print("Number of Columns:", cols)

# 5. What are the column names?
print("\nAttributes (Columns):")
print(df.columns.tolist())

# 6. Check for missing values (empty cells)
print("\nMissing Values:")
print(df.isnull().sum())    # counts empty cells PER column
print("\nTotal Missing Values:", df.isnull().sum().sum())

# 7. Basic statistics on the "G3" (final grade) column
print("\nBasic Statistics for Marks (G3):")
print("Mean Marks:", df['G3'].mean())
print("Median Marks:", df['G3'].median())
print("Minimum Marks:", df['G3'].min())
print("Maximum Marks:", df['G3'].max())

# 8. Separate columns into "Numbers" vs "Text/Categories"
numerical_cols = df.select_dtypes(include=['int64', 'float64']).columns
categorical_cols = df.select_dtypes(include=['object']).columns
print("\nNumerical Attributes:", list(numerical_cols))
print("Categorical Attributes:", list(categorical_cols))

# 9. Draw a Histogram (how many students got which mark range)
if 'G3' in df.columns:
    plt.figure(figsize=(8, 5))
    plt.hist(df['G3'], bins=10)
    plt.title("Histogram of Final Grade (G3)")
    plt.xlabel("G3 Marks")
    plt.ylabel("Frequency")
    plt.grid(True)
    plt.show()

# 10. Correlation: does one number-column move together with another?
correlation_matrix = df[numerical_cols].corr()
print("\nCorrelation Matrix:")
print(correlation_matrix)

# 11. Which columns could we PREDICT in future exercises?
possible_targets = [c for c in ['G1', 'G2', 'G3'] if c in df.columns]
print("\nPossible Target Variables:", possible_targets)

```

Bonus tip: `df.describe()` prints count, mean, std deviation, min, max, and quartiles for every numeric column — one command, instant summary!

A histogram groups numbers into "buckets" (bins) and shows how many values fall into each bucket as a bar. Tall bars = many students got that mark range.

KEY TERMS

- **.head(n)** — preview the first n rows.
- **.shape** — (rows, columns) of the table.
- **.isnull().sum()** — count of missing/empty values.
- **Correlation** — a number from -1 to 1 showing how strongly two number columns move together (1 = perfectly together, -1 = perfectly opposite, 0 = unrelated).
- **Numerical vs Categorical** — Numerical = measurable numbers (age, salary). Categorical = labels/groups (gender, city).

Part 1b — Text Data (Movie Reviews / NLP Basics)

What are we doing? We take a big pile of movie reviews and analyze the language in it — counting words, sentences, and finding the most common ones. This is the start of **Natural Language Processing (NLP)**, the field of ML dealing with human text.

Dataset: IMDB Dataset.csv — thousands of movie reviews in a review column

Library	Why We Need It
pandas	Load the CSV and grab the review column.
nltk	Split text into sentences/words, remove common "filler" words.
collections.Counter	Quickly count how many times each word appears.
wordcloud	Draw a picture where frequently-used words appear bigger.
matplotlib.pyplot	Display the word cloud image.

One-time setup — run this once before first use:

```
import nltk
nltk.download('punkt')
nltk.download('punkt_tab')
nltk.download('stopwords')
```

FULL CODE

```

import pandas as pd
import nltk
from nltk.corpus import stopwords
from nltk.tokenize import sent_tokenize, word_tokenize
from collections import Counter
from wordcloud import WordCloud
import matplotlib.pyplot as plt

# 1. Load the dataset and combine ALL reviews into one giant block of text
df = pd.read_csv(r"D:\SEM_5\ML\datasets\Text_data\IMDB Dataset.csv")
reviews = df['review']
text = " ".join(reviews.astype(str)) # glue every review together with spaces

# 2. Count sentences ("tokenize" = split text into pieces)
sentences = sent_tokenize(text) # splits text at ".", "!", "?" etc.
print("\nNumber of Sentences:", len(sentences))

# 3. Count words
words = word_tokenize(text)
words = [word.lower() for word in words if word.isalpha()] # keep only real words, lowercase
print("\nNumber of Words:", len(words))

# 4. Find the 10 most common words
word_freq = Counter(words)
print("\nTop 10 Most Frequent Words:")
for word, count in word_freq.most_common(10):
    print(f"{word} : {count}")

# 5. Remove "stop words" (common filler words like "the", "is", "and")
stop_words = set(stopwords.words('english'))
filtered_words = [word for word in words if word not in stop_words]
filtered_freq = Counter(filtered_words)
print("\nTop 10 Words After Removing Stop Words:")
for word, count in filtered_freq.most_common(10):
    print(f"{word} : {count}")

# 6. Vocabulary size = number of UNIQUE words used
vocabulary = set(filtered_words)
print("\nVocabulary Size:", len(vocabulary))

# 7. Average sentence length (words per sentence)
avg_sentence_length = len(words) / len(sentences)
print("\nAverage Sentence Length:", round(avg_sentence_length, 2), "words")

# 8. Draw a Word Cloud (bigger word = used more often)
wordcloud = WordCloud(width=1000, height=500, background_color='white').generate(" ".join(filtered_words))
plt.figure(figsize=(12, 6))
plt.imshow(wordcloud, interpolation='bilinear')
plt.axis("off")
plt.title("Word Cloud of IMDb Reviews")
plt.show()

```

PLOT EXPLAINED — WORD CLOUD

The more often a word appears in the text, the bigger it's drawn. It's a quick visual summary of "what is this text mostly about?"

KEY TERMS

- **Tokenization** — splitting text into smaller pieces (sentences or words).
- **Stop words** — very common words ("the", "a", "is") with little meaning, often removed before analysis.
- **Vocabulary** — the set of all unique words used.

- **Corpus** — an NLP term meaning "a collection of text."

Part 1c — Image Data (CIFAR-10 Pictures)

What are we doing? A digital image is really just a grid of numbers. Each pixel has 3 numbers for Red, Green, Blue (RGB) brightness (0-255). We load one image, view it, and inspect its numbers.

Dataset: CIFAR-10 — small 32×32 pixel color photos, stored using Python's pickle format

Library	Why We Need It
pickle	CIFAR-10 files are saved in "pickle" format — reads them back into Python.
numpy	Reshape and rearrange raw pixel numbers into a proper image grid.
matplotlib.pyplot	Actually display the image on screen.

FULL CODE

```

import pickle
import numpy as np
import matplotlib.pyplot as plt

# A small helper function to open a pickle file and load its contents
def unpickle(file):
    with open(file, 'rb') as fo:          # 'rb' = read binary mode
        data = pickle.load(fo, encoding='bytes')
    return data

# 1. Load a batch of images
batch = unpickle(r"D:\SEM_5\ML\datasets\Image_data\cifar-10-batches-py\data_batch_1")
image = batch[b'data'][0]                # grab just the FIRST image (stored as one long list)

# CIFAR-10 stores images as 3072 numbers = 3 channels x 32 height x 32 width, flattened
image = image.reshape(3, 32, 32)         # reshape back into (channels, height, width)
image = np.transpose(image, (1, 2, 0))    # rearrange to (height, width, channels)

# 2. Display the image
plt.imshow(image)
plt.title("CIFAR-10 Image")
plt.axis('off')
plt.show()

# 3. Check the image's dimensions
height, width, channels = image.shape
print("Image Dimensions:")
print("Height =", height)
print("Width =", width)
print("Channels =", channels)            # 3 channels = Red, Green, Blue (color image)

# 4. Is it color (RGB) or black-and-white (Grayscale)?
if channels == 3:
    print("\nImage Type: RGB")
    gray = np.mean(image, axis=2)        # average the R,G,B values per pixel
    plt.imshow(gray, cmap='gray')
    plt.title("Grayscale Image")
    plt.axis('off')
    plt.show()
else:
    print("\nImage Type: Grayscale")

# 5. Total number of pixels
pixels = height * width
print("\nTotal Number of Pixels =", pixels)

```

PLOT EXPLAINED

`plt.imshow()` displays an array of numbers as a picture. We show it twice: once in full color (RGB), once in grayscale (by averaging the 3 color numbers per pixel into a single "brightness" number).

KEY TERMS

- **Pixel** — one single dot/square in a digital image.
- **Channel** — one "layer" of color info. RGB images have 3 channels. Grayscale has 1.
- **Reshape** — rearranging a list of numbers into a different grid/array shape without changing the numbers themselves.

Data Cleaning

What are we doing? Real datasets are messy — missing cells, duplicate rows, weird spelling ("Male", "male", " MALE ") and freak values (like a salary of ₹99,999,999). Data cleaning fixes all this before building a model, because **"garbage in = garbage out."**

Two datasets are cleaned here: (a) an HR Employee dataset, (b) a messy Cafe Sales dataset.

Library	Why We Need It
pandas	Load, inspect, and clean the tables — the only library needed for this exercise.

Part 2a — Cleaning Employee Data

FULL CODE

```

import pandas as pd

# 1. Load the dataset
df = pd.read_csv(r"D:\SEM_5\ML\datasets\Data cleaning\employee\WA_Fn-UseC_-HR-Employee-Attrition.csv")
print("Original Shape:", df.shape)
records_before = len(df)

# 2. Find and FIX missing values
print("\n===== MISSING VALUES =====")
print(df.isnull().sum())
missing_before = df.isnull().sum().sum()

for col in df.columns:
    if df[col].dtype == "object":          # text/category column
        df[col] = df[col].fillna(df[col].mode()[0])    # fill blanks with MOST COMMON value
    else:                                  # number column
        df[col] = df[col].fillna(df[col].median())    # fill blanks with the MIDDLE value
missing_after = df.isnull().sum().sum()

# 3. Remove duplicate rows (exact copies)
duplicate_count = df.duplicated().sum()
df = df.drop_duplicates()
print("\nDuplicates Removed:", duplicate_count)

# 4. Standardize messy text values (e.g. "male", "MALE", " Male " -> "Male")
if "Gender" in df.columns:
    df["Gender"] = (
        df["Gender"].astype(str).str.strip().str.lower()
        .replace({"male": "Male", "female": "Female"})
    )
if "Department" in df.columns:
    df["Department"] = df["Department"].astype(str).str.strip().str.title()

# 5. Make sure number columns are actually stored as numbers
print("\n===== DATA TYPES BEFORE =====")
df.info()
for col in df.columns:
    if "Age" in col or "Income" in col or "Rate" in col or "Years" in col:
        df[col] = pd.to_numeric(df[col], errors="coerce") # convert; failures become NaN
print("\n===== DATA TYPES AFTER =====")
df.info()

# 6. Remove OUTLIERS (freakishly high/low salaries) using the IQR method
salary_col = "MonthlyIncome"
Q1 = df[salary_col].quantile(0.25)    # value below which 25% of data falls
Q3 = df[salary_col].quantile(0.75)    # value below which 75% of data falls
IQR = Q3 - Q1                         # the "middle spread" of the data
lower_bound = Q1 - (1.5 * IQR)
upper_bound = Q3 + (1.5 * IQR)
outliers_removed = len(df[(df[salary_col] < lower_bound) | (df[salary_col] > upper_bound)])
df = df[(df[salary_col] >= lower_bound) & (df[salary_col] <= upper_bound)]

# 7. Save the cleaned dataset to a new file
df.to_csv("cleaned_employee.csv", index=False)
records_after = len(df)

# 8. Print a clean summary of everything we did
print("\n===== CLEANING SUMMARY =====")
print("Records Before Cleaning :", records_before)
print("Records After Cleaning  :", records_after)
print("Missing Values Found    :", missing_before)
print("Missing Values Removed   :", missing_before - missing_after)
print("Duplicates Removed      :", duplicate_count)
print("Salary Outliers Removed  :", outliers_removed)

```

KEY TERMS

- **Missing value / NaN** — an empty cell in the data.
- **Mode** — the most frequently occurring value (used for filling missing text).
- **Median** — the middle value when numbers are sorted (used for missing numbers, since it isn't thrown off by outliers the way "mean" can be).
- **Outlier** — a value unusually far from the rest of the data.
- **IQR (Interquartile Range)** — $Q3 - Q1$, the "typical spread" of the middle 50% of data. Values far outside this range are flagged as outliers.

Part 2b — Cleaning a Messy Cafe Sales Dataset

Same idea as 2a, but with extra problems: numbers stored as text, invalid transactions (negative quantities), and broken dates.

[FULL CODE](#)

```

import pandas as pd

# 1. Load Dataset
df = pd.read_csv(r"D:\SEM_5\ML\datasets\Data cleaning\Dirty Cars\archive\dirty_cafe_sales.csv")
print("Original Shape:", df.shape)
records_before = len(df)

# 2. Peek at basic info
print("\nDataset Info")
df.info()

# 3. Fill missing TEXT values with the most common value in that column
print("\nMissing Values")
print(df.isnull().sum())
missing_before = df.isnull().sum().sum()
for col in df.columns:
    if df[col].dtype == "object":
        df[col] = df[col].fillna(df[col].mode()[0])
missing_after = df.isnull().sum().sum()
missing_removed = missing_before - missing_after

# 4. Remove duplicate transactions
duplicates_removed = df.duplicated().sum()
df = df.drop_duplicates()

# 5. Force these columns to be numeric (some entries may be garbage text like "N/A")
df["Quantity"] = pd.to_numeric(df["Quantity"], errors="coerce")
df["Price Per Unit"] = pd.to_numeric(df["Price Per Unit"], errors="coerce")
df["Total Spent"] = pd.to_numeric(df["Total Spent"], errors="coerce")

# Fill any resulting blanks with the median (middle value)
df["Quantity"] = df["Quantity"].fillna(df["Quantity"].median())
df["Price Per Unit"] = df["Price Per Unit"].fillna(df["Price Per Unit"].median())
df["Total Spent"] = df["Total Spent"].fillna(df["Total Spent"].median())

# 6. Remove invalid transactions (quantity or price can't be zero/negative)
invalid_transactions = len(df[(df["Quantity"] <= 0) | (df["Price Per Unit"] <= 0)])
df = df[(df["Quantity"] > 0) & (df["Price Per Unit"] > 0)]

# 7. Fix the date column, and drop rows where the date couldn't be understood
df["Transaction Date"] = pd.to_datetime(df["Transaction Date"], errors="coerce")
invalid_dates = df["Transaction Date"].isnull().sum()
df = df.dropna(subset=["Transaction Date"])

# 8. Standardize text formatting
df["Item"] = df["Item"].astype(str).str.strip().str.lower()
df["Payment Method"] = df["Payment Method"].astype(str).str.strip().str.title()
df["Location"] = df["Location"].astype(str).str.strip().str.title()

# 9. Save the cleaned file
df.to_csv("cleaned_sales_dataset.csv", index=False)
records_after = len(df)

# 10. Summary
print("\n===== CLEANING SUMMARY =====")
print("Records Before Cleaning      :", records_before)
print("Records After Cleaning         :", records_after)
print("Missing Values Removed          :", missing_removed)
print("Duplicate Records Removed       :", duplicates_removed)
print("Invalid Transactions Removed    :", invalid_transactions)
print("Invalid Dates Removed           :", invalid_dates)

```

KEY TERMS

- **pd.to_numeric(..., errors="coerce")** — try to convert a column to numbers; anything that fails becomes missing (NaN) instead of crashing the program.
- **pd.to_datetime(...)** — convert text like "2024-05-01" into a real date Python understands.
- **.str.strip()** / **.str.lower()** / **.str.title()** — remove extra spaces / lowercase / Title Case — used to standardize messy text.

Data Preparation (Encoding & Scaling)

What are we doing? ML models only understand numbers — they can't read text like "Male" or "Chennai." This exercise converts categories into numbers (**encoding**) and rescales number columns onto a similar range (**scaling**), which helps most models perform better.

Dataset: emp-data.xlsx — an employee Excel file (Age, Gender, Department, Salary, etc.)

Library	Why We Need It
pandas	Load the Excel file, view/manage the table.
matplotlib.pyplot	Plot histograms/boxplots to visualize the effect of scaling.
sklearn.preprocessing	LabelEncoder, StandardScaler, MinMaxScaler — convert text to numbers and rescale numeric columns.
openpyxl (behind the scenes)	Needed by pandas to read .xlsx Excel files — install with <code>pip install openpyxl</code> .

Concepts First

- Label Encoding** — gives each category a number: Male → 0, Female → 1. Good for categories with only 2 options, or a natural order (Low/Medium/High). Risky for unordered categories with 3+ values, since the model might wrongly think 2 > 1 > 0 means something.
- One-Hot Encoding** — creates a new column per category, filled with 1 (yes) or 0 (no). E.g., Department becomes Department_Sales, Department_HR, Department_IT. Safer for unordered categories.
- StandardScaler** — rescales a column so its mean becomes 0 and standard deviation becomes 1. Good for bell-curve-shaped data.
- MinMaxScaler** — rescales a column so all values fit between 0 and 1. Good when you want a fixed, bounded range.

FULL CODE

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.preprocessing import LabelEncoder, StandardScaler, MinMaxScaler

# Load Dataset
df = pd.read_excel(r"D:\SEM_5\ML\datasets\Data Preperation\emp-data.xlsx")
print("===== ORIGINAL DATASET =====")
print(df.head())

# Task 1: Identify numeric vs categorical columns
numerical_columns = df.select_dtypes(include=['int64', 'float64']).columns
categorical_columns = df.select_dtypes(include=['object']).columns
print("\nNumerical Attributes:", list(numerical_columns))
print("Categorical Attributes:", list(categorical_columns))

# Task 2: Label Encoding (turns text categories into numbers 0,1,2...)
label_encoder = LabelEncoder()
df_label = df.copy() # always copy before changing, so original stays intact
df_label["Gender"] = label_encoder.fit_transform(df_label["Gender"])
df_label["Education"] = label_encoder.fit_transform(df_label["Education"])
print("\nAfter Label Encoding:")
print(df_label[["Gender", "Education"]].head())
```

```

# Task 3: One-Hot Encoding (turns each category into its own 0/1 column)
df_onehot = pd.get_dummies(df_label, columns=["Department", "City", "Work_Mode", "Job_Role"])
print("\nAfter One-Hot Encoding:")
print(df_onehot.head())

# Task 4: StandardScaler (mean = 0, standard deviation = 1)
standard_df = df_onehot.copy()
standard_scaler = StandardScaler()
columns_to_scale = ["Age", "Experience", "Salary", "Performance_Score"]
standard_df[columns_to_scale] = standard_scaler.fit_transform(standard_df[columns_to_scale])
print("\nAfter Standard Scaling:")
print(standard_df[columns_to_scale].head())

# Task 5: MinMaxScaler (squeezes values into the 0-1 range)
minmax_df = df_onehot.copy()
minmax_scaler = MinMaxScaler()
minmax_df[columns_to_scale] = minmax_scaler.fit_transform(minmax_df[columns_to_scale])
print("\nAfter Min-Max Scaling:")
print(minmax_df[columns_to_scale].head())

# Task 6: Visualize the EFFECT of scaling on the "Salary" column
plt.figure(figsize=(10, 4))
plt.hist(df["Salary"], bins=10)
plt.title("Salary Before Standard Scaling")
plt.xlabel("Salary"); plt.ylabel("Frequency")
plt.show()

plt.figure(figsize=(10, 4))
plt.hist(standard_df["Salary"], bins=10)
plt.title("Salary After Standard Scaling")
plt.xlabel("Scaled Salary"); plt.ylabel("Frequency")
plt.show()

plt.figure(figsize=(6, 4))
plt.boxplot(df["Salary"])
plt.title("Salary Before Standard Scaling")
plt.show()

plt.figure(figsize=(6, 4))
plt.boxplot(standard_df["Salary"])
plt.title("Salary After Standard Scaling")
plt.show()

print("\nInterpretation:")
print("StandardScaler changes the SCALE of Salary.")
print("The SHAPE of the distribution stays the same.")
print("Mean becomes approx 0, standard deviation becomes 1.")

# Task 7: Compare Label Encoding vs One-Hot Encoding side by side
department_label = df.copy()
department_label["Department"] = LabelEncoder().fit_transform(department_label["Department"])
print("\nLabel Encoding Example:")
print(department_label[["Department"]].head())

department_onehot = pd.get_dummies(df[["Department"]], columns=["Department"])
print("\nOne-Hot Encoding Example:")
print(department_onehot.head())

```

PLOTS EXPLAINED

Histogram (before vs after scaling) — scaling changes the numbers on the x-axis, but the overall shape of the distribution stays the same — it's just relabeled onto a new scale.

Boxplot — a compact chart showing minimum, 25th percentile, median, 75th percentile, maximum, and

outliers (dots) of a column in one glance. The "box" is the middle 50% of the data (the IQR from Exercise 2!).

KEY TERMS

- **Encoding** — converting text categories into numbers.
- **Scaling / Normalization** — adjusting the range of number columns so no single column dominates just because its numbers happen to be bigger (e.g., Salary in the 10,000s vs Age in the 10s-90s).
- **fit_transform()** — "learn the transformation rule from this data, AND apply it immediately."

Principal Component Analysis (PCA)

What are we doing? Imagine a dataset with 20 columns — too many to plot or visualize easily. PCA squeezes many columns down into just a few new "super-columns" (called **principal components**) that still capture most of the important patterns. This is called **dimensionality reduction**.

Dataset: The classic Iris flower dataset — 4 measurements for 3 flower species, built into scikit-learn

Library	Why We Need It
<code>pandas</code>	View the data as a table.
<code>matplotlib.pyplot</code>	Scatter-plot the 2 new PCA columns.
<code>sklearn.datasets.load_iris</code>	Loads the built-in Iris dataset — no download needed.
<code>sklearn.preprocessing.StandardScaler</code>	PCA works best when all columns are on the same scale first.
<code>sklearn.decomposition.PCA</code>	The actual PCA algorithm.

FULL CODE

```

import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA

# 1. Load the Iris dataset (already built into scikit-learn)
iris = load_iris()
X = iris.data           # the 4 measurement columns
y = iris.target         # the flower species (0, 1, or 2)
df = pd.DataFrame(X, columns=iris.feature_names)

# 2. Peek at the data
print(df.head())

# 3. Standardize (scale) all 4 features first — PCA is sensitive to scale!
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# 4. Apply PCA, asking it to compress the 4 columns down to just 2
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)

# 5. View the new, smaller dataset
pca_df = pd.DataFrame(X_pca, columns=["PC1", "PC2"])
print(pca_df)

# 6. How much "information" (variance) does each new component capture?
print(pca.explained_variance_ratio_)

# 7. Total information retained after compressing 4 columns into 2
total_variance = pca.explained_variance_ratio_.sum()
print("Total Variance Retained:", total_variance * 100, "%")

# 8. Compare the shape before and after
print("Original Shape:", X.shape)           # (150 rows, 4 columns)
print("Reduced Shape:", X_pca.shape)        # (150 rows, 2 columns)

# 9. Scatter plot: PC1 vs PC2, colored by flower species
plt.figure(figsize=(8, 6))
colors = ['red', 'green', 'blue']
for i in range(3):
    plt.scatter(
        X_pca[y == i, 0],      # all PC1 values for species i
        X_pca[y == i, 1],      # all PC2 values for species i
        color=colors[i],
        label=iris.target_names[i]
    )
plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.title("PCA of Iris Dataset")
plt.legend()
plt.grid(True)
plt.show()

```

PLOT EXPLAINED — PCA SCATTER PLOT

Each dot is one flower, plotted using its 2 new "super-columns" (PC1, PC2) instead of the original 4 measurements. If the 3 colors (species) form clearly separate clusters, it means those 2 components captured enough information to tell the species apart — proving PCA successfully compressed the data without losing important patterns.

KEY TERMS

- **Dimensionality Reduction** — reducing the number of columns/features while keeping most of the useful information.
- **Principal Component (PC)** — a new column created by PCA; a mathematical mix of the original columns.
- **Explained Variance Ratio** — how much of the original data's "spread"/information each principal component captures.

Decision Tree Classifier

What are we doing? A Decision Tree learns a series of yes/no questions (like "Is the passenger male? → Is their class 3?") to arrive at a prediction — like a flowchart. We predict whether a Titanic passenger survived.

Dataset: The Titanic dataset, loaded from seaborn's built-in sample datasets

Library	Why We Need It
pandas	Handle the table of passengers.
seaborn	Instantly load the built-in titanic dataset with <code>sns.load_dataset('titanic')</code> .
sklearn.model_selection.train_test_split	Split data into a training set (to learn from) and a test set (to check performance).
sklearn.preprocessing.LabelEncoder	Convert text columns (like "male"/"female") into numbers.
sklearn.tree	DecisionTreeClassifier, plot_tree — build and draw the decision tree model.
sklearn.metrics.accuracy_score	Measure what percentage of predictions were correct.
matplotlib.pyplot	Draw the tree diagram.

FULL CODE

```
import pandas as pd
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import accuracy_score
import matplotlib.pyplot as plt

# 1. Load the Titanic dataset (built into seaborn)
df = sns.load_dataset('titanic')

# 2. Quick exploration (EDA)
print(df.head())
print("\nRows and Columns:", df.shape)
print("\nTarget Column: survived")
print(df.info())
print(df.isnull().sum())

# 3. Keep only the columns we want to use
df = df[['sex', 'pclass', 'age', 'sibsp', 'parch', 'fare', 'embarked', 'survived']]

# 4. Drop rows with any missing values (simplest cleaning approach here)
df = df.dropna()

# 5. Convert text columns to numbers
le = LabelEncoder()
df['sex'] = le.fit_transform(df['sex']) # male/female -> 0/1
df['embarked'] = le.fit_transform(df['embarked']) # port letters -> numbers

# 6. Separate features (X) from target (y)
```

```

X = df.drop('survived', axis=1)      # everything EXCEPT the answer column
y = df['survived']                  # the answer column (0 = died, 1 = survived)

# 7. Split into training data (80%) and testing data (20%)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
# random_state=42 makes the "random" split reproducible every time you run the code

# 8. Build and train the Decision Tree
model = DecisionTreeClassifier(
    criterion='entropy',    # how the tree decides which question is "best" at each step
    max_depth=4,           # limits how deep the tree can go (prevents overfitting)
    random_state=42
)
model.fit(X_train, y_train)        # LEARN the patterns from training data

# 9. Predict on the unseen test data
y_pred = model.predict(X_test)

# 10. Check accuracy
print("\nAccuracy:", accuracy_score(y_test, y_pred))

# 11. Draw the tree
plt.figure(figsize=(18, 10))
plot_tree(
    model,
    feature_names=X.columns,
    class_names=['Died', 'Survived'],
    filled=True,            # colors boxes by predicted class
    rounded=True,
    fontsize=9
)
plt.show()

# 12. Predict for one brand-new, made-up passenger
new_data = pd.DataFrame({
    'sex': [1], 'pclass': [3], 'age': [25],
    'sibsp': [0], 'parch': [0], 'fare': [15.5], 'embarked': [2]
})
prediction = model.predict(new_data)
print("\nPrediction:", "Survived" if prediction[0] == 1 else "Died")

```

PLOT EXPLAINED — DECISION TREE DIAGRAM

`plot_tree()` draws the tree as a flowchart. Each box is a "question" (e.g., "sex <= 0.5?"). Following Yes/No branches down leads you to a leaf box showing the final prediction, colored by which class it predicts.

KEY TERMS

- **Classification** — predicting a category (Survived / Died), as opposed to a number.
- **Entropy** — a measure of "messiness"/uncertainty in a group of data. The tree picks questions that reduce entropy the most.
- **max_depth** — a limit on how many levels of questions the tree can ask. Smaller = simpler tree, less overfitting risk.
- **random_state** — a fixed "seed" number so random operations give the same result every time you re-run the code.
- **Overfitting** — when a model learns training data too specifically and performs poorly on new data.

Simple Linear Regression

What are we doing? Regression predicts a **number** (not a category). Here we predict Salary from Years of Experience, by fitting the "best straight line" through the data points — the classic $y = mx + c$ from school math, but the computer finds m (slope) and c (intercept) for us.

Dataset: Salary_Data.csv — YearsExperience and Salary columns

Library	Why We Need It
pandas	Load and view the CSV.
numpy	Used for np.sqrt() (square root) to calculate RMSE, and to generate synthetic data.
matplotlib.pyplot	Plot the data points and the fitted regression line.
sklearn.model_selection.train_test_split	Split into training/testing sets.
sklearn.linear_model.LinearRegression	The regression model itself.
sklearn.metrics	mean_absolute_error, mean_squared_error, r2_score — measure how good the model's predictions are.

OPTIONAL — GENERATE THE SAMPLE DATASET

```
import pandas as pd
import numpy as np

# For reproducible results
np.random.seed(42)      # makes the "random" numbers reproducible

# Generate Years of Experience (0.5 to 20 years)
years_experience = np.round(np.linspace(0.5, 20, 50), 1)

# Generate Salary
# Formula: Salary = 30000 + (9500 * YearsExperience) + Random Noise
salary = 30000 + (9500 * years_experience) + np.random.randint(-5000, 5000, size=50)

# Create DataFrame
df = pd.DataFrame({
    "YearsExperience": years_experience,
    "Salary": salary
})

# Save to CSV
df.to_csv("Salary_Data.csv", index=False)
print("Dataset Generated Successfully!\n")
print(df.head())
```

FULL CODE — LINEAR REGRESSION

```

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

# Load Dataset
df = pd.read_csv("Salary_Data.csv")
print("Dataset")
print(df.head())

# Select Independent (X) and Dependent (y) Variables
X = df[['YearsExperience']] # double brackets keep it as a table (2D), which sklearn requires
y = df['Salary']

# Train-Test Split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Build Linear Regression Model
model = LinearRegression()
model.fit(X_train, y_train)

# Model Parameters
slope = model.coef_[0]
intercept = model.intercept_
print("\nSlope:", slope)
print("Intercept:", intercept)

# Prediction
y_pred = model.predict(X_test)

# Performance Metrics
mae = mean_absolute_error(y_test, y_pred) # average size of the errors
mse = mean_squared_error(y_test, y_pred) # average of squared errors (punishes big misses)
rmse = np.sqrt(mse) # square root of MSE, back in original units
r2 = r2_score(y_test, y_pred) # how much of the pattern the model explains

print("\nPerformance Metrics")
print("-----")
print("MAE :", mae)
print("MSE :", mse)
print("RMSE:", rmse)
print("R2 Score:", r2)

# Best Fitting Line Equation
print("\nBest Fitting Line Equation")
print(f"Salary = {slope:.2f} * YearsExperience + {intercept:.2f}")

# Plot Regression Line
plt.figure(figsize=(8, 5))
plt.scatter(X, y, color='blue', label='Actual Data')
plt.plot(X, model.predict(X), color='red', linewidth=2, label='Regression Line')
plt.xlabel("Years of Experience")
plt.ylabel("Salary")
plt.title("Simple Linear Regression")
plt.legend()
plt.show()

```

PLOT EXPLAINED — SCATTER + REGRESSION LINE

Blue dots = real salary data points. The red line is the model's "best guess" straight line through those

points. The closer the dots hug the line, the better the model fits the data.

KEY TERMS

- **Regression** — predicting a continuous number (as opposed to a category).
- **Slope (m)** — how much y (Salary) changes for every 1-unit increase in x (Experience).
- **Intercept (c)** — the predicted value of y when $x = 0$.
- **MAE** — Mean Absolute Error: average of $|\text{actual} - \text{predicted}|$. Same units as the target, easy to interpret.
- **MSE / RMSE** — Mean Squared Error and its square root; penalize large errors more heavily than MAE.
- **R² Score** — a value from 0 to 1 showing how well the model explains the variation in the data. 1.0 = perfect predictions.

Decision Tree Pruning & Evaluation Metrics

What are we doing? This exercise compares an **unrestricted** (fully-grown) Decision Tree against a **pruned** (limited-depth) Decision Tree on a synthetic "will the student Pass or Fail?" dataset, and studies evaluation metrics in depth — not just accuracy, but Precision, Recall, F1-score, Specificity, and the Confusion Matrix.

Library	Why We Need It
pandas	Build a DataFrame from generated data.
numpy	Randomly generate the synthetic student data.
sklearn.model_selection.train_test_split	Split into training/testing.
sklearn.tree.DecisionTreeClassifier	Build both the full and pruned trees.
sklearn.metrics	accuracy_score, precision_score, recall_score, f1_score, confusion_matrix, classification_report — evaluate the models in multiple ways.

FULL CODE

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score,
    f1_score, confusion_matrix, classification_report
)

np.random.seed(42)
n = 1000

# 1. Generate a fake dataset of 1000 students
study = np.random.randint(1, 11, n)      # study hours per day (1-10)
att = np.random.randint(50, 101, n)      # attendance % (50-100)
assign = np.random.randint(0, 21, n)      # assignment score (0-20)
internal = np.random.randint(0, 51, n)    # internal exam marks (0-50)

# A made-up formula decides the final "score"
score = (
    study * 3 +
    att * 0.3 +
    assign * 2 +
    internal * 1.5
)
result = np.where(score >= 90, "Pass", "Fail") # threshold rule for pass/fail

df = pd.DataFrame({
    "study": study, "att": att, "assign": assign,
    "internal": internal, "result": result
})
print("Dataset Sample")
print(df.head())

# 2. Convert Pass/Fail text to 1/0
df["result"] = df["result"].map({"Fail": 0, "Pass": 1})
X = df.drop("result", axis=1)
```

```
y = df["result"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# ----- MODEL 1: Unrestricted (full-grown) tree -----
full_tree = DecisionTreeClassifier(random_state=42)      # no depth limit -> grows until "pure"
full_tree.fit(X_train, y_train)
pred_full = full_tree.predict(X_test)

print("\nUNRESTRICTED DECISION TREE")
print("Accuracy :", accuracy_score(y_test, pred_full))
print("Precision:", precision_score(y_test, pred_full))
print("Recall   :", recall_score(y_test, pred_full))
print("F1 Score :", f1_score(y_test, pred_full))

cm = confusion_matrix(y_test, pred_full)
tn, fp, fn, tp = cm.ravel()      # unpack the 4 numbers from the 2x2 confusion matrix
spec = tn / (tn + fp)           # specificity formula
print("Specificity:", spec)
print("\nConfusion Matrix\n", cm)
print("\nClassification Report\n", classification_report(y_test, pred_full))

# ----- MODEL 2: Pre-pruned tree (limited depth) -----
pruned_tree = DecisionTreeClassifier(max_depth=4, random_state=42)  # forced to stay simple
pruned_tree.fit(X_train, y_train)
pred_pruned = pruned_tree.predict(X_test)

print("\nPRE-PRUNED DECISION TREE (max_depth=4)")
print("Accuracy :", accuracy_score(y_test, pred_pruned))
print("Precision:", precision_score(y_test, pred_pruned))
print("Recall   :", recall_score(y_test, pred_pruned))
print("F1 Score :", f1_score(y_test, pred_pruned))

cm = confusion_matrix(y_test, pred_pruned)
tn, fp, fn, tp = cm.ravel()
spec = tn / (tn + fp)
print("Specificity:", spec)
print("\nConfusion Matrix\n", cm)
print("\nClassification Report\n", classification_report(y_test, pred_pruned))
```

The Confusion Matrix, Explained Simply

	Predicted: Fail	Predicted: Pass
Actual: Fail	True Negative (TN)	False Positive (FP)
Actual: Pass	False Negative (FN)	True Positive (TP)

- **Accuracy** = (correct predictions) / (all predictions) — overall correctness.
- **Precision** = of everyone the model said "Pass," how many actually passed? $TP / (TP + FP)$
- **Recall (Sensitivity)** = of everyone who actually passed, how many did the model catch? $TP / (TP + FN)$
- **F1 Score** = a balance between Precision and Recall (their harmonic mean) — useful when you care about both.
- **Specificity** = of everyone who actually failed, how many did the model correctly say "Fail"? $TN / (TN + FP)$

PRUNING, EXPLAINED

An unrestricted tree keeps splitting until it perfectly separates the training data — this often means it overfits (memorizes noise, performs worse on new data). A pruned tree (here, `max_depth=4`) is deliberately kept simpler, usually generalizing better to unseen data even if training accuracy looks slightly lower.

KEY TERMS

- **Pruning** — limiting a tree's growth (either during training with `max_depth`, or after training by trimming branches) to reduce overfitting.
- **Confusion Matrix** — a table comparing actual vs predicted classes, the foundation for most classification metrics.

Logistic Regression

What are we doing? Despite the name, Logistic Regression is used for **classification**, not number prediction — it predicts the probability that something belongs to a category (e.g., "78% chance this passenger survived"), then converts that into a Yes/No decision. Here we predict Titanic survival again, but with a different type of model than Exercise 5.

Library	Why We Need It
pandas, numpy	Data handling and math.
seaborn	Load the built-in titanic dataset.
matplotlib.pyplot	Draw the accuracy bar chart.
sklearn.preprocessing.LabelEncoder	Convert text columns to numbers.
sklearn.model_selection.train_test_split	Split into training/testing.
sklearn.linear_model.LogisticRegression	The classification model.
sklearn.metrics	accuracy_score, confusion_matrix, classification_report — evaluate performance.

FULL CODE

```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

# Load Dataset
df = sns.load_dataset('titanic')
print("First 5 rows")
print(df.head())
print("\nDataset Shape:", df.shape)

# Select useful features
df = df[['survived', 'pclass', 'sex', 'age', 'sibsp', 'parch', 'fare', 'embarked']]

# Handle Missing Values (fill instead of dropping, to keep more data)
df['age'] = df['age'].fillna(df['age'].median())
df['embarked'] = df['embarked'].fillna(df['embarked'].mode()[0])

# Convert Categorical Columns
encoder = LabelEncoder()
df['sex'] = encoder.fit_transform(df['sex'])
df['embarked'] = encoder.fit_transform(df['embarked'])

# Split Features and Target
X = df.drop('survived', axis=1)
y = df['survived']

# Train-Test Split
X_train, X_test, y_train, y_test = train_test_split(
```

```

X, y, test_size=0.2, random_state=42
)

# Train Logistic Regression Model
model = LogisticRegression(max_iter=1000)    # max_iter = give the model more attempts to find the fit
model.fit(X_train, y_train)

# Predictions
y_pred = model.predict(X_test)

# Accuracy on training data vs testing data
train_acc = model.score(X_train, y_train)
test_acc = model.score(X_test, y_test)
print("\nTraining Accuracy :", round(train_acc*100, 2), "%")
print("Testing Accuracy  :", round(test_acc*100, 2), "%")

# Confusion Matrix and Classification Report
print("\nConfusion Matrix")
print(confusion_matrix(y_test, y_pred))
print("\nClassification Report")
print(classification_report(y_test, y_pred))

# Bar chart comparing Training vs Testing accuracy
accuracy = [train_acc*100, test_acc*100]
labels = ['Training Accuracy', 'Testing Accuracy']
plt.figure(figsize=(6, 5))
plt.bar(labels, accuracy)
plt.ylabel("Accuracy (%)")
plt.title("Training vs Testing Accuracy")
plt.ylim(0, 100)
for i, v in enumerate(accuracy):
    plt.text(i, v + 1, f"{v:.2f}%", ha='center', fontsize=11)
plt.show()

```

PLOT EXPLAINED — TRAINING VS TESTING ACCURACY BAR CHART

Two bars comparing how well the model did on data it learned from (Training) vs data it had never seen (Testing). If Training accuracy is much higher than Testing accuracy, that's a sign of overfitting.

KEY TERMS

- **Logistic Regression** — a classification algorithm that outputs a probability (0 to 1) and applies a threshold (usually 0.5) to decide the final class.
- **max_iter** — the maximum number of optimization attempts the algorithm makes while learning; increase it if the model complains it "didn't converge."

Handling Imbalanced Data with SMOTE

What are we doing? Some real-world problems are imbalanced — e.g., in fraud detection, 95% of transactions are legitimate and only 5% are fraud. A model can get "95% accuracy" just by always guessing "not fraud" — which is useless! **SMOTE** (Synthetic Minority Over-sampling Technique) fixes this by creating new, realistic synthetic examples of the rare class so the model actually learns to recognize it.

Library	Why We Need It
pandas	Build a DataFrame from the generated data.
matplotlib.pyplot, seaborn	Plot class distribution charts (before/after SMOTE).
sklearn.datasets.make_classification	Generate a synthetic imbalanced dataset for practice.
sklearn.model_selection.train_test_split	Split into training/testing.
sklearn.linear_model.LogisticRegression	The classification model used to test the effect of SMOTE.
sklearn.metrics	classification_report, confusion_matrix — evaluate performance before/after balancing.
imblearn.over_sampling.SMOTE	The library that actually creates the synthetic minority samples.

Install this extra library first: `pip install imbalanced-learn`

FULL CODE

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, confusion_matrix
from imblearn.over_sampling import SMOTE

# STEP 1: Generate a synthetic, IMBALANCED "Fraud Detection" dataset
# weights=[0.95, 0.05] means: 95% legit transactions, only 5% fraud
X, y = make_classification(
    n_samples=5000,
    n_features=4,
    n_informative=2,
    n_redundant=0,
    n_clusters_per_class=1,
    weights=[0.95, 0.05],
    random_state=42,
    flip_y=0.01
)
df = pd.DataFrame(X, columns=['Transaction_Amount', 'Time_of_Day', 'User_Age', 'Distance_from_Home'])
df['Is_Fraud'] = y

# STEP 2: Visualize the imbalance
plt.figure(figsize=(6, 4))
sns.countplot(data=df, x='Is_Fraud', hue='Is_Fraud', palette='Set2', legend=False)
plt.title('Class Distribution Before SMOTE (Imbalanced)')
plt.xlabel('Class (0 = Legitimate, 1 = Fraud)')
plt.ylabel('Count')
plt.show()
```

```

print("Class counts before SMOTE:")
print(df['Is_Fraud'].value_counts())

# Split into train/test. stratify=y keeps the SAME imbalance ratio in both splits
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y
)

# STEP 3: Train a BASELINE model (no fix for imbalance) to see the problem
print("\n--- BASELINE MODEL (Before SMOTE) ---")
baseline_model = LogisticRegression(random_state=42)
baseline_model.fit(X_train, y_train)
y_pred_baseline = baseline_model.predict(X_test)
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred_baseline))
print("\nClassification Report:\n", classification_report(y_test, y_pred_baseline))

# STEP 4: Apply SMOTE -- but ONLY to the training data (never touch the test data!)
smote = SMOTE(random_state=42)
X_train_smote, y_train_smote = smote.fit_resample(X_train, y_train)

# Visualize the NEW, balanced training data
plt.figure(figsize=(6, 4))
sns.countplot(x=y_train_smote, hue=y_train_smote, palette='Set1', legend=False)
plt.title('Class Distribution After SMOTE (Balanced Training Data)')
plt.xlabel('Class (0 = Legitimate, 1 = Fraud)')
plt.ylabel('Count')
plt.show()

print("\nClass counts in Training Set AFTER SMOTE:")
print(pd.Series(y_train_smote).value_counts())

# STEP 5: Train a NEW model on the balanced data, and compare results
print("\n--- SMOTE MODEL (After SMOTE) ---")
smote_model = LogisticRegression(random_state=42)
smote_model.fit(X_train_smote, y_train_smote)
y_pred_smote = smote_model.predict(X_test)
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred_smote))
print("\nClassification Report:\n", classification_report(y_test, y_pred_smote))

```

PLOT EXPLAINED — CLASS DISTRIBUTION COUNT PLOT

`sns.countplot()` draws a bar for each class showing how many rows belong to it. The "Before SMOTE" chart shows one tall bar (legit) and one tiny bar (fraud) — visual proof of imbalance. The "After SMOTE" chart (on the training set only) shows both bars roughly equal — SMOTE generated new synthetic fraud examples to match.

Very important rule: SMOTE is applied only to the training data, never to the test data. The test set must reflect the real, imbalanced world so we get an honest performance check.

KEY TERMS

- **Class Imbalance** — when one category has far fewer examples than another in a dataset.
- **SMOTE** — creates new synthetic (not copied) examples of the minority class by interpolating between existing minority examples and their neighbors.
- **stratify=y** — ensures the train/test split keeps the same class proportions as the original data.
- **Data Leakage** — accidentally letting information from the test set influence training (e.g., applying SMOTE before splitting) — this makes performance look better than it really is. Always split first, then balance only the training set.

Plotting Quick Reference

Every chart type used across the 9 exercises, in one table.

Chart	Function	Best For	Used In
Histogram	plt.hist()	Showing the spread/distribution of ONE number column	Ex 1, 3
Word Cloud	WordCloud().generate() + plt.imshow()	Visualizing which words are most common in text	Ex 1
Image display	plt.imshow()	Showing an actual picture from pixel data	Ex 1
Boxplot	plt.boxplot()	Showing spread + outliers of ONE number column	Ex 3
Scatter plot	plt.scatter()	Showing the relationship between TWO number columns	Ex 4, 6
Decision Tree diagram	plot_tree()	Visualizing how a decision tree makes decisions	Ex 5
Line plot	plt.plot()	Drawing a fitted line/trend over data	Ex 6
Bar chart	plt.bar()	Comparing a few values side by side (e.g., accuracy)	Ex 8
Count plot	sns.countplot()	Counting how many rows fall into each category	Ex 9

General Plotting Recipe (Reuse This Everywhere)

```
plt.figure(figsize=(width, height)) # 1. open a new blank canvas
plt.hist(data) # 2. draw your chart (hist / scatter / bar / plot / boxplot)
plt.title("My Chart Title") # 3. add a title
plt.xlabel("X Axis Label") # 4. label the x-axis
plt.ylabel("Y Axis Label") # 5. label the y-axis
plt.legend() # 6. (optional) show a legend if multiple series
plt.grid(True) # 7. (optional) light grid lines
plt.show() # 8. display it!
```

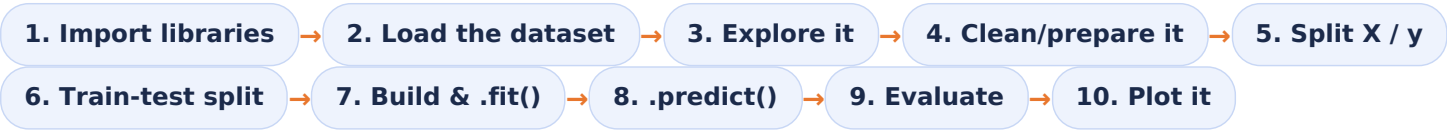
Final Exam Summary Table

#	Exercise	Type of Problem	Main Algorithm/Tool	Dataset
1	Explore Table/Text/Image data	Exploratory Data Analysis	pandas, nltk, pickle	Student marks, IMDB reviews, CIFAR-10
2	Data Cleaning	Preprocessing	pandas	HR Employee, Cafe Sales
3	Encoding & Scaling	Preprocessing	LabelEncoder, OneHot, StandardScaler, MinMaxScaler	Employee Excel data
4	Dimensionality Reduction	Unsupervised	PCA	Iris flowers
5	Classification	Supervised (Tree)	Decision Tree Classifier	Titanic
6	Regression	Supervised (Linear)	LinearRegression	Salary vs Experience

7	Pruning + Metrics	Supervised (Tree)	Decision Tree (full vs pruned)	Synthetic student pass/fail
8	Classification	Supervised (Logistic)	LogisticRegression	Titanic
9	Imbalanced Data	Preprocessing + Classification	SMOTE + Logistic Regression	Synthetic fraud detection

Last-Minute Checklist

If your mind goes blank in the exam, remember this flow — it's the same shape every time:



3. Explore = `.head()`, `.shape`, `.isnull().sum()` · 4. Clean/prepare = fill missing, encode, scale · 9. Evaluate = accuracy/MAE/confusion matrix

Good luck with your exam! Try explaining each "What are we doing?" section out loud in your own words — if you can do that without looking, you're ready.